MACS 30111 · Break the helper

Name: _____ Date: _11/5/26__

Pencil and paper only, no laptops. Assume `import numpy as np`. *Submitted during class*

This worksheet is two-sided. Parts 3–5 are on the back.

The data: a survey. Each **row** is a respondent; each **column** is a question answered from **1 to 10**. This helper standardizes each question (column) to mean 0 and std 1:

```
1  def standardize_features(data):
2      mu_vec = data.mean(axis=0)
3      sigma_vec = data.std(axis=0)
4      return (data - mu_vec) / sigma_vec
```

Part 1. Trace it by hand

Four respondents answered two questions; on Q2, all four answered 7. Run `standardize_features(d)` in your head, line by line:

```
#             Q1  Q2
d = np.array([[1., 7.],
              [3., 7.],
              [1., 7.],
              [3., 7.]])   # 4 respondents
```

Formulas (for one column, N values)
mean: $\mu = (x_1 + x_2 + \ldots + x_N) / N$
std: $\sigma = \sqrt{[(x_1 - \mu)^2 + \ldots + (x_N - \mu)^2] / N}$
NumPy's `.std()` divides by N. Example: [1, 3, 1, 3] → μ = 2, σ = √(4/4) = 1

(a) `mu_vec` = _____ shape: _____

(b) `sigma_vec` = _____ shape: _____

(c) `data - mu_vec`

(d) the value returned

(e) What goes wrong? Does Python raise an error? _____

Part 2. Shapes and broadcasting

The full survey is `(200, 5)`: 200 respondents, 5 questions. A pilot is `(5, 5)`. A classmate changes `axis=0` to `axis=1` on lines 2 and 3. Fill in rows B–D; row A is done for you.

Rule: line shapes up on the right; a missing dimension counts as 1; each pair must be equal, or one must be 1.

Row	data.shape	axis	mu_vec.shape	Does data - mu_vec run? (Yes / Error)	If it runs, is the answer correct?
A	*(200, 5)*	*0*	*(5,)*	*Yes*	*Yes*
B	(200, 5)	1			
C	(5, 5)	0			
D	(5, 5)	1			

(f) Which row, **B**, **C** or **D**, is the most dangerous: the code **runs with no error** but returns a **wrong answer**? Circle the letter, then explain why in one sentence.

B **C** **D** _____

Part 3. Brainstorm: how else could it break?

Real survey data is messy. Come up with **three more surveys** that would make `standardize_features` crash, warn, or **quietly return a wrong answer**. Don't reuse Parts 1–2.

Think about: the **answers** (what if someone skips a question? gives an extreme answer?) · the **shape** (how many respondents or questions?) · the **type** (yes/no questions? answers typed in as text?).

#	The survey (write the array, or describe it)	What happens? (circle one)	What should the function do instead?
1		error · warning · wrong answer · works fine	
2		error · warning · wrong answer · works fine	
3		error · warning · wrong answer · works fine	

Part 4. What should a good test catch?

You're writing tests for `standardize_features` before you use it on the full survey. List **four things a good set of tests should catch**. For each one, give a small survey you'd test with and what a **correct** function should return. Use Parts 1–3 for ideas.

#	A good test should catch…	Survey you'd test with	What a correct function returns
1			
2			
3			
4			

A test is only useful if it **fails on buggy code** and **passes on correct code**. When you compare floats, "equal" means equal within rounding (`np.allclose`), not `==`.

Part 5. One sentence

"No error" doesn't mean "correct" because …

Feedback (not graded)

How helpful was this activity for understanding testing? Circle one.

1 2 3 4 5

not helpful very helpful

What questions do you still have? (about standardizing, broadcasting, testing or anything else)